Intervals

> → **Key Idea**: Sort intervals and merge overlapping ones to solve scheduling and range problems

> • **When to use**: Meeting rooms, scheduling conflicts, range merging, calendar problems

> ⊕ **Complexity**: Time O(n log n) for sorting, Space O(n)

> ⚠ **Common Pitfall**: Not sorting intervals first or incorrect overlap detection logic

Overview

Visual Example

Interval Merging Process

```
Input intervals: [[1,3], [2,6], [8,10], [15,18]]

Step 1: Sort by start time (already sorted)
[1,3] [2,6] [8,10] [15,18]

Step 2: Merge overlapping intervals
[1,3] overlaps with [2,6] → merge to [1,6]
[1,6] no overlap with [8,10] → keep separate
[8,10] no overlap with [15,18] → keep separate

Result: [[1,6], [8,10], [15,18]]

Timeline visualization:
1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
[──────────────────]               [─────────]               [──────────────]
    Merged                            Kept                      Kept
```

Meeting Room Scheduling

```
Meeting times: [(0,30), (5,10), (15,20)]

Timeline:
0    5   10   15  20    30
├────┼────┼────┼──┼─────┤
│████████████████████████│ Meeting 1: (0,30)
│     ████████           │ Meeting 2: (5,10)  ← Conflicts!
│          ████████      │ Meeting 3: (15,20) ← Conflicts!
```

```
🔥 Conflicts detected → Need multiple rooms
```

Interval Intersection

```
A: [1,3], [5,7], [9,11]
B: [2,4], [6,8], [10,12]

Visual intersection:
1   2   3   4   5   6   7   8   9  10  11  12
[─────────]         [─────────]     [─────────]  A
    [─────────]         [─────────]     [─────────]  B
    [───]               [─]             [─]         Intersections

Result: [[2,3], [6,7], [10,11]]
```

Interval problems involve working with ranges of values. The key insight is usually to sort intervals by their start time and then process them in order.

Common patterns include merging overlapping intervals, finding non-overlapping intervals, and scheduling problems.

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Problem Type	Time Complexity	Space Complexity
Merge Intervals	O(n log n)	O(n)
Insert Interval	O(n)	O(n)
Meeting Rooms	O(n log n)	O(n)
Calendar Problems	O(n) per operation	O(n)

Most interval problems require O(n log n) time due to sorting, with O(n) space for the result.

Common Problem Patterns

1. Merge Intervals

Problem: Merge all overlapping intervals.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> result = new ArrayList<>();

    for (int[] interval : intervals) {
        if (result.isEmpty() || result.get(result.size() - 1)[1] < interval[0]) {
            result.add(interval);
        } else {
            result.get(result.size() - 1)[1] =
                Math.max(result.get(result.size() - 1)[1], interval[1]);
        }
    }
    return result.toArray(new int[result.size()][]);
}
```

2. Insert Interval

Problem: Insert new interval into sorted non-overlapping intervals.

```
public int[][] insert(int[][] intervals, int[] newInterval) {
    List<int[]> result = new ArrayList<>();
    int i = 0;

    // Add all intervals before newInterval
    while (i < intervals.length && intervals[i][1] < newInterval[0]) {
        result.add(intervals[i++]);
```

```
        }
        // Merge overlapping intervals
        while (i < intervals.length && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
            newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
            i++;
        }
        result.add(newInterval);
        // Add remaining intervals
        while (i < intervals.length) result.add(intervals[i++]);

        return result.toArray(new int[result.size()][]);
    }
```

3. Meeting Rooms II (Min Rooms Required)

Problem: Find minimum meeting rooms required.

```
public int minMeetingRooms(int[][] intervals) {
    int[] starts = new int[intervals.length];
    int[] ends = new int[intervals.length];

    for (int i = 0; i < intervals.length; i++) {
        starts[i] = intervals[i][0];
        ends[i] = intervals[i][1];
    }
    Arrays.sort(starts);
    Arrays.sort(ends);

    int rooms = 0, endPtr = 0;
    for (int start : starts) {
        if (start < ends[endPtr]) rooms++;
        else endPtr++;
    }
    return rooms;
}
```

4. Non-Overlapping Intervals

Problem: Remove minimum intervals to make rest non-overlapping.

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[1] - b[1]); // Sort by end time
    int count = 0, prevEnd = Integer.MIN_VALUE;

    for (int[] interval : intervals) {
        if (interval[0] >= prevEnd) {
            prevEnd = interval[1];
```

```
            } else {
                count++; // Remove this interval
            }
        }
        return count;
    }
```

← Previous: Monotonic Stack | Back to Main Guide | Next: Top-K & Heaps →